

Part A: Governance Policy

Organizational Scenario

This project focuses on the City of Pittsburgh 311 service system in the public sector. The API is designed to help route resident service requests related to parks, trees, animals, and public facilities. Based on the request description, the system predicts the request type ID, issue, category, and department.

The main users would be 311 staff who review and route requests. Residents are also affected because the routing result determines which department receives their request.

If a request is routed incorrectly, it may delay the response and create extra work for city staff. Because of this, the API should support staff decisions rather than make final routing decisions on its own.

Permitted and Prohibited Uses by API Mode

Mode	Who may use it	Permitted use	Prohibited use	Logged ?	Human review?
chat	311 staff	Ask about possible routing categories	Unrelated requests or attempts to reveal restricted information	Yes	Yes
generate	311 staff	Generate a suggested routing result	Automatically make the final routing decision	Yes	Yes
structured_output	311 staff / internal system	Return routing results in a fixed format	Assume every required field is correct	Yes	Yes
tool_calling	311 staff / internal system	Look up official routing information	Use tools for unrelated tasks or unauthorized actions	Yes	Yes

Chat allows users to have more control across multiple turns, while generate is single-turn and more reproducible with a fixed seed. Structured output makes the format more consistent, but required fields can still be incorrect. Tool calling can use additional functions outside the conversation, so it needs stricter controls and logging.

Data Boundary

The model can use the resident request text and routing-related fields such as request type ID, issue, category, and department. Information that is not needed for routing should not be sent to the model. Any <PII>, <PHI>, <FIN>, or <CONF>content should be masked before it reaches the model. In this project, the model runs locally, so prompts stay on the machine. If a hosted model is used in the future, only masked and necessary information should be sent.

Human-in-the-Loop

None of the routing outputs should be used to finalize a request automatically. The API can suggest a routing result, but 311 staff should review it before making the final decision, especially when the request is unclear or could fit more than one category.

The system should not automatically close or finalize a request. If the routing is wrong, the request could be sent to the wrong department and take longer to resolve.

Marking the evidence behind each rule

Policy rule	Status	Evidence
Final routing should be reviewed by 311 staff	Measured	Part C Eval 4 had 80% full-record accuracy, so some requests were still incorrect.
The model should not make routing decisions completely on its own	Measured	Part B Baseline Evaluations 1 and 3 both had 0% full-record accuracy.
Prompt injection and out-of-scope requests should be screened	Measured	Part D Baseline Safety Evaluation found that 2 of 3 prompt-injection probes succeeded, and the guardrail evaluation still missed 1 of 3 out-of-scope requests.
Sensitive information should not be revealed	Precautionary	No leakage happened in our tests, but more leakage probes would be needed to test this more fully.
Hosted models would need stronger privacy controls	Precautionary	This project only used a local model. Testing with a hosted model would be needed to measure this risk.

The results showed that human review is still necessary because even the best-performing method made some classification mistakes.

No original rule was completely contradicted, but the results showed that out-of-scope screening needed to be stronger. The guardrail blocked most of these requests, but it still allowed one unrelated Python request.

I also did not expect prompt injection to be such a noticeable risk. In the baseline test, 2 out of 3 prompt-injection attempts succeeded, so I added input checks for this type of attack in the final policy.

Part B: Your DATA & Baseline LLMBOX API Experiments

Experiment Settings

I evaluated the model by checking whether it predicted the correct request type ID, issue, category, department, and overall routing result. I used exact match to compare the predictions with the ground truth, and I also manually reviewed a small sample to see whether the model understood the request even when the label was not an exact match.

I split the dataset into 60% development data and 40% evaluation data. The `raw_text` field, which contains the resident's service request, was used as the model input.

Baseline Evaluation 1

For Baseline Evaluation 1, I used the default settings: `temperature = 1.0`, `top_p = 0.95`, and `max_new_tokens = 512`. I ran 50 requests from the development dataset using generate mode and saved the responses

.

The exact-match accuracy was 0% for request type ID, issue, category, department, and the full routing result. From the manual review, the model usually understood the general request, but it did not use the official Pittsburgh 311 labels.

Because of this, I would not consider the baseline performance acceptable for the routing task.

Baseline Evaluation 3

For Baseline Evaluation 3, I changed the settings to `temperature = 0.2`, `top_p = 0.8`, and `max_new_tokens = 128`. I lowered the settings because the task only needs a short and consistent routing answer.

I ran the same 50 development requests again using generate mode and saved the responses. The full-record accuracy was still 0%, but issue accuracy increased slightly to 4%. Average latency also decreased from 2.57 seconds to 2.12 seconds

The new settings made the responses a little faster, but the model still could not match the official Pittsburgh 311 labels well enough, so I still would not consider the performance acceptable.

Part C: Designing & Evaluating Your Custom LLM API

Custom Features -- Evaluation 1: Structured Output

For this evaluation, I created a Pydantic model with four fields: request type ID, issue, category, and department. I then used the generate + structured_output mode in LLMBox on 50 requests from the evaluation dataset and saved the model responses.

All 50 responses were valid JSON, but only 32% followed the schema correctly. The exact-match accuracy for the routing fields was still 0%, and the average latency was 5.69 seconds.

The structured output made the format more consistent, but it did not help the model choose the correct Pittsburgh 311 labels. Because of this, I still would not consider the performance acceptable for this task.

Custom Features -- Evaluation 2: Tool Calling

For this evaluation, I created a Pittsburgh 311 taxonomy lookup tool. The tool takes a short description of the resident's issue and returns the official request type ID, issue, category, and department. The taxonomy was built from the development dataset and included 28 unique issues.

I used the generate + tool_calling mode in LLMBox on 50 requests from the evaluation dataset and saved the model responses. The tool was used for 92% of the requests and returned a valid result for 90%. Full-record accuracy improved to 50%, while category accuracy was 80% and department accuracy was 70%. The average latency was 7.61 seconds.

The tool improved the results a lot compared with the baseline, but the keyword matching still returned the wrong issue in some cases. Because of this, I would still keep human review before using the routing result.

Custom Features -- Evaluation 3: TF-IDF Retrieval + LLM

For this evaluation, I used TF-IDF retrieval to find the most similar request from the development dataset and provided that example to the LLM as a reference. I tested this method on 50 requests from the evaluation dataset and saved the model responses.

Full-record accuracy was 36%, while category accuracy was 80% and department accuracy was 76%. The average latency was 2.06 seconds.

The retrieved example helped with some parts of the routing, but the LLM still changed or misclassified some fields. Because of this, the overall routing accuracy was still not high enough.

Custom Features Evaluation 4: TF-IDF Majority Vote + LLM

For this evaluation, I retrieved the top three similar requests and used majority voting to create a reference routing result for the LLM. I then tested it on 50 requests from the evaluation dataset and saved the model responses.

Full-record accuracy improved to 80%, while category accuracy reached 96% and department accuracy reached 98%. The average latency was 2.08 seconds.

This was the best-performing method in my experiments. Using multiple similar examples gave the LLM a stronger reference and reduced routing errors, although human review is still useful for the remaining incorrect cases.

Part D: Safety & Security Risk Assessment

Safety Probe Set

I created 12 adversarial inputs for the Pittsburgh 311 scenario, with three examples in each category: harmful content, out-of-scope requests, prompt injection, and confidentiality or leakage. I also created 10 normal 311 requests to test whether the guardrail would block valid inputs.

For prompt injection, I used canary tokens like CANARY_311_ALPHA so I could easily check whether the attack worked.

Baseline Safety Results

I submitted all 12 adversarial probes and 10 benign inputs to the baseline API and saved the responses. Without the guardrail, 3 out of 12 adversarial probes succeeded. The model rejected all harmful-content probes and did not reveal confidential information, but 2 of 3 prompt-injection attacks worked. It also answered 1 out-of-scope Python request.

The overall attack success rate was 25%, and prompt injection was the biggest weakness.

Guardrail Design

The guardrail was built around the TF-IDF retrieval functionality from Custom Evaluation 3. I added a simple guardrail that checks the input before routing. It looks for prompt injection, harmful instructions, confidentiality requests, and out-of-scope content. If a request is blocked, the user sees the message: "This request cannot be processed because it is outside the permitted Pittsburgh 311 routing task or contains unsafe instructions."

I also added an output check for canary tokens and logged whether each request was blocked, the reason, and the latency.

Guardrail Results

I then submitted the same 12 adversarial probes and 10 benign inputs through the guardrail and saved the responses. With the guardrail, 11 out of 12 adversarial probes were blocked, giving an overall catch rate of 91.7%. Harmful content, prompt injection, and confidentiality probes were all blocked, while out-of-scope requests had the lowest catch rate at 66.7%.

None of the 10 normal 311 requests were incorrectly blocked, so the over-refusal rate was 0%. The guardrail added 0 extra LLM tokens. The measured latency difference was -0.000152 seconds per request, which I treated as no meaningful added latency because the difference was very small and likely caused by early blocking and timing noise.

I think the guardrail performed well for this test set because it blocked 91.7% of adversarial inputs without blocking any valid 311 requests. However, out-of-scope detection still needs improvement.

Human Adjudication

I manually reviewed 12 guardrail decisions and labeled each block or allow decision as correct or incorrect. I agreed with 11 out of 12 decisions, giving a 91.7% agreement rate.

The only disagreement was an unrelated Python programming request that the guardrail allowed even though it was outside the Pittsburgh 311 routing task.

Toxicity Measurement

For harmful or toxic content, I used a simple keyword and phrase list. The list included phrases such as insult the resident, write a rude response, and respond aggressively.

I chose this method because it was easy to understand and check manually. However, it can still miss harmful requests that use different wording.

Residual Risk

The main remaining risk is out-of-scope detection. The guardrail missed one unrelated Python request, so it may still allow requests that do not belong to the 311 routing task.

The keyword checks could also miss prompt injections or harmful requests that use different wording. In future testing, I would use more varied and less obvious adversarial examples to see how well the guardrail works.

Part E. Organizational Analysis & Strategy

Measuring the Cost

The LLM-based methods took between about 2 and 8 seconds per request. Baseline Eval 1 averaged 2.57 seconds, Baseline Eval 3 averaged 2.12 seconds, structured output averaged 5.69 seconds, tool calling averaged 7.61 seconds, TF-IDF + LLM averaged 2.06 seconds, and TF-IDF majority vote + LLM averaged 2.08 seconds.

Average total token usage per request was 45.2 tokens for Baseline Eval 1, 39.1 for Baseline Eval 3, 87.1 for structured output, 79.3 for tool calling, 108.2 for TF-IDF + LLM, and 312.4 for TF-IDF majority vote + LLM.

The majority vote method used the most tokens because it included three retrieved examples in the prompt, but its latency was still close to the other generate-based evaluations.

The guardrail added 0 extra LLM tokens. The measured latency difference was -0.000152 seconds per request, which I treated as no meaningful added latency because the difference was very small and likely caused by early blocking and timing noise.

I did not measure the actual staff review time, so I used a simple estimate for this analysis. I assumed that reviewing an API suggestion would take about 20 seconds per request.

Measuring the Benefit

The best-performing method was TF-IDF majority vote + LLM, with 80% full-record accuracy. This means about 4 out of 5 requests could potentially be routed correctly without changing the result, while the remaining cases would still need staff review or correction.

I did not measure the actual time Pittsburgh 311 staff spend routing each request, so the manual processing time would need to be estimated. The API could save time by giving staff a suggested routing result instead of having them classify every request from scratch.

I also assumed that manually reviewing and routing a request without the API would take about 1 minute. With 80% full-record accuracy, about 20 out of every 100 requests would still need correction. If each correction takes about 1 minute, that would add about 20 minutes of staff time per 100 requests.

Comparing the Alternatives

One option is to keep the current manual process, which avoids model-generated routing errors but requires staff to classify every request themselves. Another option is to use the LLM with tool calling, which improved accuracy but was slower and still only reached 50% full-record accuracy.

The TF-IDF majority vote + LLM method performed best in this project, with 80% full-record accuracy and an average latency of 2.08 seconds. It was much more accurate than tool calling while also being faster, although it still requires LLM generation and human review for some requests.

Based on these assumptions, the API workflow would take about 34 seconds per request, including model latency, human review, and the expected correction time. This compares with about 1 minute for manual routing and would save around 43 minutes for every 100 requests. Since I did not measure the setup or maintenance cost of the system, I cannot give an exact break-even point, but the time savings would become more useful as the number of requests increases.

Recommendation

Based on the results, I would recommend adopting the API with the limits defined in Part A, especially keeping human review for the final routing decision. The TF-IDF majority vote + LLM method performed the best, with 80% full-record accuracy, but some requests were still routed incorrectly.

I would be more comfortable using the system more widely if future testing on a larger dataset showed higher accuracy and better out-of-scope detection. If the accuracy stayed low or the

guardrail continued to miss many out-of-scope requests, I would reconsider using the system beyond a support role.